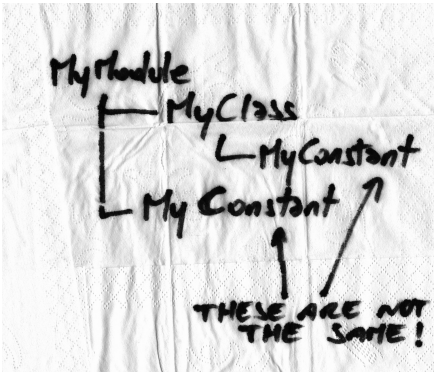
```
module MyModule
  MyConstant = 'Outer constant'

  class MyClass
    MyConstant = 'Inner constant'
  end
end
```

Bill pulls a napkin from his shirt pocket and sketches out the constants in this code. You can see the result in the following figure.



All the constants in a program are arranged in a tree similar to a file system, where modules (and classes) are *directories* and regular constants are *files*. Like in a file system, you can have multiple files with the same name, as long as they live in different directories. You can even refer to a constant by its *path*, as you'd do with a file. Let's see how.

The Paths of Constants

You just learned that constants are nested like directories and files. Also like directories and files, constants are uniquely identified by their paths. Constants' paths use a double colon as a separator (this is akin to the scope operator in C++):

```
module M
  class C
    X = 'a constant'
  end
  C::X # => "a constant"
end

M::C::X # => "a constant"
```

If you're sitting deep inside the tree of constants, you can provide the absolute path to an outer constant by using a leading double colon as root: